ChatGPT

Context-Aware Multimodal Pipeline in RAG-Anything [1]

RAG-Anything augments each document unit with its local context before indexing, so that, for example, an image or table is not processed in isolation but together with nearby text. In practice this is implemented via a **ContextExtractor** component (configurable via `RAGAnythingConfig`). During parsing, every content item (text chunk, image, table, equation, etc.) is tagged and stored with its position. The ContextExtractor then gathers surrounding text according to a user-specified *window* (e.g. neighboring pages or chunks) and mode (e.g. "page", "chunk" or token-based) [1] [2]. This textual context (including optional headings or captions) is prepended to the item before further processing. For example, an image's caption or the preceding paragraph can be included so that a vision model sees the image *and* its caption together. This "context-aware" generation step (controlled by parameters like `context_window`, `context_mode`, and `max_context_tokens`) ensures that the descriptive text produced for each multimodal item accurately reflects its role in the larger document [1] [2]. [2]

Atomic Unit Processing & Embedding [3]

After parsing, the content is split into **atomic units** by modality. Plain text is broken into chunks (typically by tokens or layout) and handled by LightRAG's text pipeline (semantic chunking, embedding, NER, relation extraction, etc.). For non-text units (images, tables, equations), RAG-Anything uses dedicated *modal processors*: e.g. an `ImageModalProcessor` uses a vision-language model (VLM) to produce a detailed caption and an "entity summary" for each figure; a `TableModalProcessor` or `EquationModalProcessor` uses LLMs to extract structured descriptions. Crucially, before calling the model, each processor invokes the ContextExtractor to prepend nearby text, headers, or captions to the input prompt. The output of this stage is twofold: (1) for each multimodal item c_j a **detailed description** d_{chunk_j} optimized for retrieval, and (2) an **entity summary** e_{entity_j} capturing key attributes (names, types, etc.) used for graph construction [1] [2]. [4]

All chunks and graph elements are then embedded: RAG-Anything builds a single **dense vector table** containing embeddings of every text chunk, every graph node, and even graph relationships. In particular, after extracting entities and relations (see below), the system embeds *all* graph vertices and edges into the unified vector space using the chosen embedding model [3]. This means that retrieval can match on entire chunks, on individual entities, or on relations, since all are represented by vectors. [5]

Knowledge Graph Construction (Entities & Links) [6]

The next stage constructs a **multimodal knowledge graph** from the processed content. Two complementary graphs are built and then fused: [7]

- **Cross-Modal Graph (Non-Text Units):** For each non-text item c_j (image, table, equation, etc.), RAG-Anything creates a multimodal entity node v_{mm_j}. It then runs an information-extraction pipeline on [8]

the item's description d_{chunk_j} to identify *intra-chunk entities* (e.g. objects in an image, variables in an equation) and their relations [4]. Those fine-grained entities V_j and relations E_j become sub-nodes and edges. Critically, each such entity is linked back to the parent item node with an explicit `belongs_to` edge. In other words, if a chart contains the entities "GDP" and "year", the graph will have nodes for those terms and edges `(GDP) -[belongs_to]-> (Chart1)` [2]. This preserves modality grounding: the image/table node serves as an anchor for its contained concepts. Thanks to the earlier context injection, the extraction also captures textual cues from surrounding context (e.g. figure captions or nearby mentions).

- **Text-Based Graph (Text Chunks):** Independently, every text chunk is processed with standard NLP (NER + relation extraction) to build a text-centric knowledge graph. This graph encodes entities and relationships found purely in the document's text. Since text already carries rich semantics, this pipeline does not need the extra cross-modal context step.

- **Graph Fusion:** The two graphs are merged by *entity alignment*: nodes from the multimodal graph and the text graph with the same or similar names are matched (e.g. the entity "enzyme" in a chart description vs "enzyme" in the body text). These alignments unify the graphs into a single knowledge graph G spanning all modalities [5] [6]. The fused graph thus encodes intra-chunk relationships (from tables/figures) and document-wide textual relations, all linked through shared entity nodes.

Retrieval Over Entities, Relations, and Chunks

During query time, RAG-Anything uses a **hybrid retrieval** strategy leveraging the unified graph and the embedding table. The query is first encoded (with awareness of any modality cues, e.g. if it mentions "figure" or "table") into the same embedding space. Then two complementary searches run in parallel:

- **Structural (Graph) Retrieval:** The system scans the knowledge graph for exact matches of entities or keywords in the query. Matching nodes are selected, and their local graph neighborhood is **expanded** (multi-hop) up to a configurable distance. Because of the `belongs_to` chains and extracted relations, this structural walk retrieves connected entities, relations, and ultimately the content chunks (text or multimodal) they belong to. For instance, a query about "fermentation" might hit an entity node in a figure, then follow a `belongs_to` edge to retrieve the figure's description and any related text chunks [2] [3].

- **Semantic (Vector) Retrieval:** Simultaneously, the query's embedding is used to perform a nearest-neighbor search in the unified embedding table. This returns the top-k semantically similar items across all modalities – whether they are whole text chunks, image captions, or even relation-descriptions. Because graph entities and relations were also embedded, the search can match on them too, surfacing relevant content that lacks explicit graph links.

The two result sets are then merged. RAG-Anything applies a **fusion scoring** that balances graph-derived signals (e.g. how central an entity is in the graph) and embedding similarity, along with any query-specified modality preferences [7] [8]. This multi-signal fusion yields a final ranked list of chunks and entity contexts, maximizing both structural relevance and semantic fit.

Benefits for Multimodal Documents [1]

By grounding every unit in context and integrating everything into one graph-plus-embedding index, RAG-Anything handles complex PDFs (with interleaved text, images, tables, etc.) far more accurately than text-only RAG. For example, an image in a PDF often comes with a caption or is referenced by a nearby paragraph; the ContextExtractor ensures the image's caption and adjacent text are included when the image is described. This disambiguates visual content (an unlabeled chart becomes meaningful when its legend is included) and links it to the proper part of the document. The graph's `belongs_to` edges mean that a query hitting an entity inside an image will still retrieve the entire figure and its caption. Likewise, any sidecar metadata (such as figure captions or surrounding headings) can be folded into context. In short, RAG-Anything's context-aware pipeline weaves together all modalities, so no relevant information is dropped. Empirically, this yields better retrieval on long documents with mixed content, because the model sees the *full multimodal context* of each piece [2] [1]. [2]

Configuration and Integration [3]

In code, context-aware processing is enabled by settings in the `RAGAnythingConfig`. By default, `context_window=1` and `context_mode="page"` (meaning each item looks at the rest of its page), but these can be tuned per dataset. Users also enable or disable each modality (images, tables, equations) in the config. The RAGAnything framework automatically wires the ContextExtractor into each multimodal processor – no extra coding is needed beyond configuration. For example: [4]

```
config = RAGAnythingConfig(
    context_window=1,     # how many pages/chunks around each item to use
    context_mode="page",  # or "chunk" for neighbor blocks, or "token" for max
tokens
    max_context_tokens=2000,
    enable_image_processing=True,
    enable_table_processing=True,
    enable_equation_processing=True,
    ...
)
rag = RAGAnything(config)
rag.index_documents([...])  # builds the graph+index using context-aware
processing
```

[5]

In practice, one simply supplies multimodal documents (PDFs, images, etc.) and RAG-Anything's end-to-end pipeline (parsing → content analysis → graph/index building → hybrid retrieval) runs with context-awareness built in. The result is a unified index where images, text, tables and their metadata are all interlinked, yielding more accurate and disambiguated RAG retrieval. [2] [3] [6]

Sources: RAG-Anything implements context-aware "local neighborhood" processing by including adjacent content in each unit's description [1] [2]. It builds dual knowledge graphs (cross-modal and text) with explicit `belongs_to` links [2] [4], then fuses them and embeds all entities/chunks [3]. Retrieval then combines graph traversals and vector search for robust multimodal matching [7] [8]. [7]

1 3 4 6 8 RAG-Anything: All-in-One RAG Framework
https://arxiv.org/html/2510.12323v1

2 5 7 RAG-Anything: Unified Multimodal RAG
https://www.emergentmind.com/papers/2510.12323